VIBE CODER & DEVELOPER ONBOARDING MANUAL

REHMIA 풀스택 스타터팩 v1.0 바이브코딩 온보딩 & 도커 배포 마스터 가이드

"이 스타터팩을 내려받은 누구나 5분 만에 로컬 IDE에서 바이브코딩을 시작하고, 1줄 명령어로 도커 배포까지 완료할 수 있는 친절한 실전 레퍼런스"

- **대상 독자:** 바이브코더(AI 페어 프로그래밍 입문자), 주니어/풀스택 개발자
- **백엔드 스택:** Spring Boot 4.0.8 / Java 21 / Spring Security 6 / JPA / QueryDSL 5.1
- **데이터베이스:** PostgreSQL 16 (운영/도커) / H2 In-Memory (테스트 격리)
- **프론트엔드:** Thymeleaf (SSR) + Tailwind CSS v4 + 비동기 공통 모듈 (Api/Toast/Modal)
- **패키징 & 배포:** Docker Multi-stage + Nginx Alpine (80포트 리버스 프록시)
- **최종 검증 상태:** 빅테크 100점 만점 기술검정 통과본 (2026-10-16 기준)

목차 (Table of Contents)

1. **Part 1. [초보자 퀵스타트] 5분 만에 내 컴퓨터/IDE에서 바로 시작하기** ("이것만 고치면 바로 돌아갑니다")
2. **Part 2. [바이브코딩 핵심 규칙] AI와 함께 코드를 뜯어고칠 때 지켜야 할 규격** (Toast, Modal, Api, DTO 표준)
3. **Part 3. [도커 & 클라우드 배포] 1줄 명령어로 완성하는 상용 패키징** (`docker-compose`, Nginx, 환경변수)
4. **Part 4. [도메인 핵심 엔진] 이커머스 트랜잭션 & 2027 브랜드 스토리 뷰** (재고 자동화, RBAC, 패럴랙스)
5. **Part 5. [상황별 장애 조치 & 트러블슈팅 FAQ]** (자주 겪는 포트 충돌, DB 연결 실패, 이미지 엑스박스 해결)

> 💡 **바이브코더를 위한 핵심 안내:** 본 스타터팩은 네이버/카카오/라인 테크 리드 기준의 엄격한 기술 검증을 거쳐 **100점 만점을 획득한 안정적인 풀스택 베이스**입니다. 여러분은 복잡한 스프링 시큐리티, CSRF 방어, 이미지 업로드 파이프라인, 도커 인프라를 처음부터 만들 필요 없이, AI에게 *"이 스타터팩에 쿠폰 기능 추가해줘"*, *"리뷰 게시판 달아줘"*라고 지시하기만 하면 됩니다!

Part 1. [초보자 퀵스타트] 5분 만에 내 IDE에서 바로 시작하기

이 스타터팩을 ZIP으로 다운로드받았거나 Git Clone 한 직후, **"딱 3가지만 확인/수정"**하면 내 컴퓨터에서 즉시 실행할 수 있습니다.

1. 사전 준비물 (Prerequisites)

- **JDK 21 필수:** Eclipse Temurin 21 또는 Oracle OpenJDK 21이 설치되어 있어야 합니다.
- **IDE 권장:** IntelliJ IDEA (Ultimate 또는 Community) 권장. (VS Code도 가능)
- **PostgreSQL 16:** 로컬 PC에 설치되어 있거나, 도커로 띄울 수 있어야 합니다.

2. 🚨 "이것만 고치고 시작하세요!" (3대 필수 설정)

1 DB 비밀번호 맞추기 (`src/main/resources/application.properties`)

내 컴퓨터에 설치된 PostgreSQL의 `shopdb` 데이터베이스 및 비밀번호와 일치시킵니다.

```
# application.properties (7~9번째 줄)
spring.datasource.url=jdbc:postgresql://localhost:5432/shopdb
spring.datasource.username=postgres
spring.datasource.password=내_DB_비밀번호  # ← 여기를 내 비밀번호로 수정!
```

※ PostgreSQL에 아직 DB가 없다면 pgAdmin이나 터미널에서 `CREATE DATABASE shopdb;` 명령을 1회 실행하세요.

2 상품 이미지 저장 폴더 1초 생성 (`C:/shopmall/item`)

관리자가 상품을 등록할 때 업로드된 이미지가 저장되는 로컬 폴더입니다. Windows 탐색기에서 아래 폴더를 미리 생성해 주세요:

```
📂 생성할 폴더 경로: C:\shopmall\item
```

※ Mac 또는 Linux 사용자라면 `application.properties` 에서 `itemImgLocation=/Users/내이름/shopmall/item` 으로 변경하시면 됩니다.

3 초기 테스트 계정 확인 (`InitData.java`)

서버를 처음 켜면 `src/main/java/com/example/shop01/config/InitData.java` 가 자동으로 관리자/고객 계정과 8종의 상품 데이터를 DB에 넣어줍니다. 바로 로그인해서 테스트할 수 있습니다:

역할 (Role)	로그인 아이디 (이메일)	비밀번호	이용 가능 기능
시스템 관리자	`admin@shop01.com`	`1234`	상품 등록(`/admin/item/new`), 상품 관리 및 재고 수정 (`/admin/items`)
일반 테스트 고객	`user@shop01.com`	`1234`	상품 탐색, 장바구니 담기, 바로 주문, 구매 이력 조회 및 주문 취소

3. 실행하기 (Run & Verify)

IntelliJ에서 실행

1. IntelliJ에서 프로젝트 루트 폴더를 **Open**합니다.
2. 우측 상단 `Shop01Application.java`를 찾아 ▶ **Run** 버튼을 누릅니다.
3. 웹 브라우저를 열고 `http://localhost:8080` 에 접속하면 하이엔드 르미아 쇼핑몰이 즉시 열립니다!

4. 테스트 스위트 무결성 검증 (`./gradlew test`)

프로젝트를 수정하기 전, 터미널에서 아래 명령을 실행하여 모든 서비스/리포지토리 테스트가 정상 통과하는지 확인하세요:

```
# Windows PowerShell 환경
.\gradlew.bat test

# Mac / Linux 환경
./gradlew test
```

✅ **안심하고 테스트하세요:** 본 프로젝트의 테스트 환경(`src/test/resources/application.properties`)에는 독립된 **H2 In-Memory DB (PostgreSQL 호환 모드)**가 내장되어 있어, 실제 로컬 PostgreSQL 데이터를 건드리지 않고 100% 독립 격리되어 초고속 실행됩니다.

Part 2. [바이브코딩 핵심 규칙] AI와 함께 코드를 고칠 때 지켜야 할 규격

AI(ChatGPT, Claude, Gemini, Cursor 등)에게 새 기능 개발을 요청할 때, 아래 3가지 규격을 지시문에 함께 제공하면 **절대 코드가 깨지지 않고 고품질의 엔터프라이즈 코드가 작성**됩니다.

1. 🎨 프론트엔드 모듈 사용법 (브라우저 Native alert/confirm 사용 금지!)

본 스타터팩은 브라우저 팝업(`alert` , `confirm`)이 100% 퇴출되어 있으며, 모든 뷰에 `Toast` , `Modal` , `Api` 전용 모듈이 자동 탑재되어 있습니다.

TOAST 감성 알림 토스트 (`toast.js`)

우측 상단에 3초간 노출되는 부드러운 다크 알림

```
// 성공 안내 (초록)
Toast.success('쇼핑백에 담았습니다.');

// 오류 안내 (빨강)
Toast.error('상품 재고가 부족합니다.');

// 일반 안내 (노랑)
Toast.info('로그인이 필요한 서비스입니다.');
```

MODAL 비동기 컨펌 모달 (`modal.js`)

Promise 기반으로 동작하는 세련된 커스텀 모달

```
// 사용자 확인 대기 (비동기)
const ok = await Modal.confirm('정말 삭제하시겠습니까?', {
    title: '상품 삭제 확인',
    confirmText: '삭제하기',
    type: 'danger' // 빨간색 강조
});
if (!ok) return; // 취소 시 중단
```

🌐 표준 비동기 API 통신 클라이언트 (`api.js`)

Spring Security CSRF 토큰을 자동으로 헤더에 넣어주고, 중복 클릭 방지 로딩 스피너를 버튼에 자동으로 띄워줍니다:

```
// 버튼 요소를 넘겨주면 클릭 시 자동으로 "처리 중..." 스피너가 돌며 중복 클릭 방지!
const res = await Api.post('/order', { itemId: 1, count: 2 }, myButtonElement);

if (res.success) {
    Toast.success(res.message); // 서버 메시지 출력
    setTimeout(() => location.href = '/orders', 500);
} else {
    Toast.error(res.message);   // 서버 에러 메시지 자동 표출
}
```

2. 🛡️ 백엔드 표준 API 응답 규격 (`ApiResponse`)

새로운 비동기 Controller 엔드포인트를 만들 때는 임의의 문자열을 반환하지 말고, 항상 `ApiResponse` 로 감싸서 반환하세요:

```
// OrderController 예시
@PostMapping("/order")
@ResponseBody
public ResponseEntity<ApiResponse<Long>> order(@RequestBody @Valid OrderDto orderDto, ...) {
    Long orderId = orderService.order(orderDto, email);
    // 일관된 JSON 반환: { "success": true, "message": "주문 완료", "data": 1001 }
    return ResponseEntity.ok(ApiResponse.success(orderId, "주문이 정상 접수되었습니다."));
}
```

3. 🧩 글로벌 예외 처리기 (`GlobalExceptionHandler.java`)

Controller 안에서 복잡하게 `try-catch` 를 일일이 쓰지 마세요. 비즈니스 로직(Service)에서 `throw new IllegalStateException("재고 부족");` 을 던지면, 글로벌 핸들러가 자동으로 가로채어 클라이언트에게 깨끗한 400 Bad Request JSON 에러를 내려줍니다.

> 💡 **AI 프롬프트 팁:** *"이 기능 추가할 때 프론트엔드는 alert() 쓰지 말고 Toast.success()와 Modal.confirm(), Api.post()를 쓰고, 백엔드는 ApiResponse<T> 규격과 GlobalExceptionHandler를 따르도록 작성해줘."*라고 주문하세요.

Part 3. [도커 & 클라우드 배포] 1줄 명령어로 완성하는 상용 패키징

본 프로젝트는 **DB(PostgreSQL) + 백엔드 앱(Spring Boot 4) + 프론트 웹서버(Nginx)**가 완벽하게 통합된 Docker Compose 오케스트레이션을 제공합니다.

1. 3단 컨테이너 아키텍처 구조도

서비스명	사용 이미지	외부 포트	주요 역할 및 최적화
nginx	`nginx:alpine`	`80`	• 전면 리버스 프록시 • Gzip 텍스트/CSS/JS 압축 전송 • 업로드 이미지(`/images/**`) 볼륨 직통 30일 브라우저 캐싱
app	멀티스테이지 경량 빌드	8080 (내부)	• Eclipse Temurin 21 JRE Alpine 기반 경량화 • non-root 보안 유저(`spring`) 실행 • JVM G1GC 메모리 최적화 (`MaxRAMPercentage=75.0`)
db	`postgres:16-alpine`	`5432`	• PostgreSQL 16 데이터베이스 • `pg_isready` 헬스체크 (DB 준비 완료 후 App 구동 보장) • Named Volume을 통한 데이터 영속 보존

2. 원클릭 도커 배포 3단계 순서

1 환경변수 파일 생성 (`.env`)

프로젝트 루트에 기본 `.env` 파일이 이미 준비되어 있습니다. 운영 서버 비밀번호를 바꾸려면 `.env` 파일을 열어 수정하세요:

```
POSTGRES_DB=shopdb
POSTGRES_USER=postgres
POSTGRES_PASSWORD=내_운영_비밀번호
APP_PORT=80
```

2 도커 컴포즈 빌드 & 기동 (명령어 1줄!)

터미널에서 아래 명령을 실행하면 이미지 다운로드, JAR 컴파일, Nginx 설정이 모두 전자동으로 실행됩니다:

```
docker-compose up -d --build
```

3 접속 및 운영 관리

- **서비스 접속:** 브라우저에서 `http://서버IP` 또는 `http://localhost` 접속 (80 포트 Nginx 자동 연결)
- **실시간 로그 확인:** `docker-compose logs -f app`
- **서비스 중지:** `docker-compose down`

> 💾 **데이터 영속성 보장 (Volume):** 컨테이너를 재시작하거나 내려도(`docker-compose down`), PostgreSQL 데이터(`shop_db_data`)와 업로드된 상품 사진(`shop_media_data`)은 독립된 Docker 볼륨에 안전하게 보존됩니다.

Part 4. [도메인 핵심 엔진] 이커머스 트랜잭션 & 2027 브랜드 스토리

1. 2-Track 듀얼 코어 기획 구조

TRACK A **E-Commerce 코어 트랜잭션 엔진**

메인 홈, 상품 상세, 장바구니, 주문 결제, 관리자 센터

- Spring Security 6 RBAC 기반 폼/비동기 인증 및 CSRF 방어
- QueryDSL 5.1 기반 다중 동적 필터링 및 페이징 검색
- 주문-재고 실시간 자동 차감/복구 원자적 트랜잭션
- 5슬롯 다중 이미지 자동 파이프라인 (1번 슬롯 대표 썸네일)

TRACK B **2027 Interactive Brand Experience**

URL: `/story` (르미아 브랜드 스토리텔링 뷰)

- Obsidian Dark & Sand White 럭셔리 미니멀리즘 디자인
- Scroll-Driven Dynamic 자간 확장 애니메이션
- Centered Scale-on-Scroll 뷰포트 고정 줌인 캔버스
- 마우스 커서 실시간 추적 마그네틱 앰비언트 글로우
- 5단계 퀵 점프 내비게이터 및 실시간 스크롤 게이지

2. 주문-재고 실시간 트랜잭션 무결성 규칙

상황	시스템 자동 처리 내용	비즈니스 예외 및 롤백 규칙
주문 접수 (Order)	주문 수량만큼 DB 재고 즉시 자동 차감	주문 수량 > 재고 시 `IllegalStateException` 발생, 주문 트랜잭션 전체 롤백
주문 취소 (Cancel)	취소된 수량만큼 DB 재고 즉시 원상복구	주문 상태가 `CANCEL` 로 변경되며 본인 주문만 취소 가능 (403 방어)
장바구니 주문 (Cart Order)	선택된 상품들만 일괄 주문 후 해당 카트 비우기	선택 상품 중 1개라도 재고가 부족하면 전체 주문 실패 및 롤백 (원자성 보장)

Part 5. [상황별 장애 조치 & 트러블슈팅 FAQ]

Q1. "Connection refused" 데이터베이스 연결 실패 오류가 발생합니다.

원인: 로컬 PostgreSQL 서비스가 실행 중이지 않거나, `shopdb` 데이터베이스가 생성되지 않았습니다.
해결 방법:

1. Windows 서비스 관리자(services.msc)에서 `postgresql-x64-16` 서비스가 "실행 중"인지 확인하세요.
2. 도커를 사용 중이라면 `docker-compose up -d db` 명령으로 DB 컨테이너만 먼저 띄울 수 있습니다.
3. `application.properties` 의 `spring.datasource.password` 가 내 DB 비밀번호와 일치하는지 확인하세요.

Q2. 업로드한 상품 이미지가 엑스박스(깨짐)로 나옵니다.

원인: 이미지를 저장할 로컬 폴더가 컴퓨터에 실제로 생성되어 있지 않습니다.
해결 방법:

Windows 내 컴퓨터에서 `C:\shopmall\item` 폴더가 실제로 존재하는지 확인하고, 없다면 **새 폴더로 직접 생성**하세요. (서버는 폴더가 없으면 이미지를 저장하지 못합니다.)

Q3. 8080 포트가 이미 사용 중(Address already in use)이라고 뜹니다.

원인: 오라클 DB(XDB), 다른 스프링부트 프로세스, 톰캣 등이 8080 포트를 점유하고 있습니다.
해결 방법:

`src/main/resources/application.properties` 파일 맨 밑에 아래 한 줄을 추가하여 포트를 변경하세요:
`server.port=8081` (접속 주소: `http://localhost:8081`)

Q4. 터미널에서 `./gradlew` 실행 시 JAVA_HOME 에러가 발생합니다.

원인: 컴퓨터의 환경변수에 JDK 21 경로가 등록되지 않았습니다.
해결 방법:

```
# PowerShell에서 1회성 임시 지정 후 실행
$env:JAVA_HOME="C:\Users\<사용자이름>\.jdks\temurin-21.0.12.1"
.\gradlew.bat test
```

Q5. 새 HTML 화면을 만들었는데 Tailwind CSS 디자인이 적용되지 않아요.

해결 방법:

본 프로젝트는 Tailwind CLI 컴파일 방식(`static/css/output.css`)을 사용합니다. 새 클래스를 많이 추가했다면 프로젝트 루트에서 `npm run build` 를 실행하여 CSS를 재컴파일하거나, 개발 중에는 `npm run dev` (자동 감지 빌드)를 켜두세요.

REHMIA ATHLETIC FULL-STACK STARTER PACK v1.0

Production-Ready Enterprise Architecture & Vibe Coding Reference Guide

기술 지원 및 문의: support@rehmia-athletic.com | 르미아 크리에이티브 스튜디오